

**Singapore Institute of Technology
Information and Communications Technology Cluster**

ICT4011 Capstone Project

Capstone Project Progress Report Submission Form (Form E1)

A. Preamble

This form duly filled and signed must be attached to each Capstone Project Progress Report submitted.

Reporting Period:

(Reporting Period): 1/9/2025 to 5/12/2025

Candidate Particulars:

(Name of Candidate): Tay Siang Long

(Organization): ST Engineering Urban Solutions Ltd.

(Project Period): 5/5/2025 to 10/4/2026

Main Academic Supervisor Details:

(Name): Spiridon Bakiras

(Email Address): spiridon.bakiras@singaporetech.edu.sg

(Contact Number): 65923324

(Designation): Associate Professor

Industry Supervisor Details:

(Name): Wu Yingjie

(Email Address): yingjie.wu@stengg.com

(Contact Number): 88517312

(Designation): Assistant Principal Engineer

(Department / Division): URS

B. Declaration of Conformity

B.1 Declaration by Industry Supervisor

I hereby acknowledge that I have read and understood the contents of the Capstone Project Progress Report, and deem the contents appropriate for release to the Singapore Institute of Technology for use in the assessment of the candidate.

Signature



Name of Industry Supervisor: Wu Yingjie
Designation: Assistant Principal Engineer
Department: URS
Contact No: 88517312
Email: yingjie.wu@stengg.com
Date: 3/12/2025

B.2 Declaration by Candidate

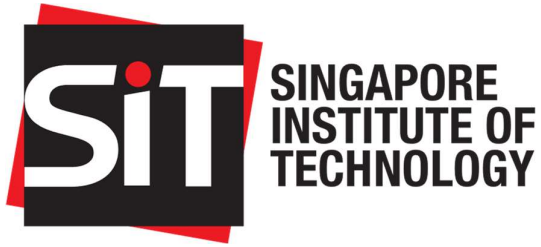
I hereby acknowledge that I have engaged and discussed with my Industry Supervisor on the contents of the Capstone Project Progress Report, and have sought approval for the release of the report to the Singapore Institute of Technology.

Signature



Name of Candidate: Tay Siang Long
Contact No: 81618762
Email: 2203190@sit.singaporetech.edu.sg
Date: 1/12/2025

END OF FORM E



Information and Communications Technology Cluster

Multi-Agent System for Query-to-Dashboard

**Capstone Project Progress Report
(ST Engineering Urban Solutions Ltd.)**

For the reporting period from 1/9/2025 to 5/12/2025

**Tay Siang Long
2203190**

Name of Industry Supervisor: Wu YingJie

Name of Academic Supervisor: Spiridon Bakiras

Submitted as part of the requirement for ICT4011 Capstone Project

(Body Text, 16, Bold, Center) Capstone Project Progress Report for the Period of 1/9/2025, to 5/12/2025

1. Introduction

This project focuses on developing an intelligent Analysis Agent capable of transforming natural-language questions into structured analytical outputs such as descriptive summaries, statistical results, and visualizations. The new version of the agent has undergone a major architectural shift compared to the prototype described in the previous trimester.

The system is now built around a finite-state workflow using LangGraph, with a local Qwen3-14B-GGUF model serving as the primary reasoning engine. It integrates a FastMCP tool server for modular analytics functions, maintains a session-aware memory, and communicates through an A2A-compliant message protocol.

This progress report reflects the evolution, technical achievements, challenges, and knowledge gained during this trimester, written in the same structure as the previous E1 but presenting the updated system design.

2. Literature Review

ReAct and LangGraph Workflows

ReAct (“Reasoning + Acting”) introduces a pattern where an LLM alternates between thinking and executing actions [1]. This improves multi-step reliability and makes the reasoning process interpretable. LangGraph extends this idea by giving developers state-based control: each step of the agent’s reasoning cycle (planning, tool execution, observation, looping) is represented as a node in a graph.

This is well-suited for data analytics, where tasks must follow predictable sequences (e.g., clean → analyze → visualize). Recent extensions like Pre-Act show that structured, stepwise workflows improve tool-use consistency [2].

Our agent adopts this model: the LLM plans the next step, the graph determines the allowed transitions, and the system invokes analysis tools only when appropriate. This ensures reliability and transparency which are two qualities essential for analytical pipelines.

Local LLM Inference and GGUF

Running the agent locally is now feasible due to improved quantization techniques that compress LLM weights. GGUF, used by llama.cpp, is a binary format optimized for local inference with minimal memory overhead [4]. Research shows quantized models maintain strong performance despite reduced precision [3].

However, quantized weights can be vulnerable to injection attacks if not validated [6]. This trade-off highlights the value of local inference while emphasizing the need for careful deployment.

Our system uses Qwen3-14B-GGUF locally. This supports fast inference and full data sovereignty which is important for applications involving sensitive datasets.

Modular Pipelines and MCP Tooling

A major trend in LLM-agent research is relying on modular tool pipelines. Surveys describe how agents choose tools, invoke them, and integrate results into their reasoning [7]. In enterprise settings, multi-agent systems generate dashboards using this approach, as seen in Zhang & Elhamod's Data-to-Dashboard framework [8].

Our design incorporates these principles using a FastMCP server, which exposes analysis utilities (cleaning, summary statistics, anomaly detection, plotting). The agent dynamically selects tools during its reasoning loop based on user intent. This modularity allows rapid extension without modifying core agent logic.

Memory and Persistent Analytical Context

Analytical workflows often require multi-step refinement. A session-aware memory helps agents avoid repeating previous calculations and ensures continuity. Research identifies the lack of long-term memory as a limitation in current tool-using agents [9].

Inspired by this, our system includes a Memory Store that logs previous queries, tool results, and summary insights in structured form. This enables context-aware responses across a session.

Multi-Agent Communication and Open Protocols

As agents scale, structured communication becomes essential. Kandogan et al. outline the need for clear task-IDs, capabilities registries, and result schemas in enterprise-grade "Compound AI" systems [10].

Our architecture aligns with these principles. The A2A-compliant service sends and receives JSON messages that include:

- task_id,
- query,
- result,
- status,
- and timestamps.

This ensures every result can be traced and every workflow is auditable.

3. Planned Tasks and Objectives

The primary objectives for this trimester were:

1. Convert the old agent into a finite-state graph structure using LangGraph.
2. Replace cloud LLMs with a local Qwen3-14B-GGUF model for privacy and offline capability.
3. Enhance the tool orchestration layer using FastMCP for modular analytics.
4. Introduce session-aware memory for persistent context.
5. Implement an A2A-style communication protocol for agent-to-orchestrator interaction.
6. Demonstrate end-to-end analysis: natural-language query → planning → tool execution → visualization → report.

These tasks aimed to create a more robust, maintainable, and extensible analysis agent suitable for real-world data workflows.

4. Accomplishments During the Reporting Period

4.1 Built a Deterministic State-Graph Workflow

I redesigned the agent into a clear, step-by-step state machine. Previously, the LLM decided how the workflow should move forward, which caused loops and inconsistent behavior. Each step now has clear input/output rules, which makes failures much easier to locate. The state graph agent governs transitions such as:

1. Receive Query → Plan
2. Plan → Tool Selection
3. Tool Selection → Tool Execution
4. Tool Execution → Observation
5. Observation → Plan (loop) or Final Summary

Each state has a defined input and expected output, making the entire process easier to follow and debug. The shared state object looks like this:

```
class AgentState(BaseModel):  
    query: str  
    plan: str | None = None  
    tool: str | None = None  
    observation: str | None = None  
    result: str | None = None
```

Figure 1. Code snippet showing simplified version of agent state

Because the graph controls transitions and not the LLM, this removed loops, confusion, and inconsistent tool calls. This structure ensures that every step of the workflow has context and that errors are isolated to one specific stage rather than propagating unpredictably which made debugging significantly easier.

4.2 Integrated Qwen3-14B-GGUF as the local LLM

The agent now uses a locally hosted Qwen3-14B-GGUF model instead of a cloud API.

This decision gave several benefits:

- privacy: data never leaves the machine
- reliability: no API limits or downtime
- predictable performance: inference behavior stays consistent
- lower long-term cost

The model is loaded through ChatOllama:

```
llm = ChatOllama(model="Qwen2.5:14b", options={"temperature": 0})
```

Figure 2. Code snippet showing local llm model

To keep runtime manageable and improve responsiveness, I used:

- warm-up prompting at startup,
- conservative temperatures,
- and hardware-aware quantization.

This kept the model responsive while remaining fully offline.

4.3 Added dynamic tool orchestration with FastMCP

The agent now discovers tools at runtime from a FastMCP tool server, allowing the system to be extended without modifying core logic.

```

async def discover_tools(mcp_client) -> Dict[str, Dict[str, Any]]:
    """Query MCP server and build a name-descriptor map."""
    raw = await mcp_client.list_tools() # placeholder API
    return {t["name"]: t for t in raw}

def resolve_tool(candidate: Optional[str], tool_map: Dict[str, Any]) -> Optional[str]:
    """Resolve requested tool name to an available tool (exact / case-insensitive / fuzzy)."""
    if not candidate:
        return None
    if candidate in tool_map:
        return candidate
    low_map = {k.lower(): k for k in tool_map}
    if candidate.lower() in low_map:
        return low_map[candidate.lower()]
    choices = difflib.get_close_matches(candidate.lower(),
list(low_map.keys()), n=1, cutoff=0.6)
    return low_map[choices[0]] if choices else None

async def call_tool_safe(mcp_client, tool_name: str, args: Dict[str, Any],
timeout: int = 30):
    """Invoke a tool with timeout and normalized result shape."""
    try:
        coro = mcp_client.invoke_tool(tool_name, args) # placeholder API
        result = await asyncio.wait_for(coro, timeout)
        return {"status": "ok", "tool": tool_name, "output": result}
    except asyncio.TimeoutError:
        return {"status": "error", "tool": tool_name, "error": "timeout"}
    except Exception as e:
        return {"status": "error", "tool": tool_name, "error": str(e)}

def map_plan_to_tools(plan_steps: List[Dict[str, Any]], tool_map: Dict[str, Any]) -> List[Dict[str, Any]]:
    """Attach a resolved tool name to each plan step when possible."""
    mapped = []
    for step in plan_steps:
        candidate = step.get("tool") or step.get("intent")
        resolved = resolve_tool(candidate, tool_map)
        mapped.append(**step, "resolved_tool": resolved)
    return mapped

```

Figure 3. Code snippet demonstrate runtime MCP tool discovery and invocation

This means if a new plotting function or data-cleaning tool is added to the server, the agent can use it immediately. This is a major shift from last trimester's version, which hard-coded every tool into the agent.

Having the LLM generate a plan and then map these steps to available MCP tools gives the system flexibility similar to modular software used in real data engineering teams.

4.4 Implemented Session Memory for Multi-Step Analysis

Because many analytics tasks involve follow-up questions or related queries, I added a lightweight session memory system. It stores compact entries such as:

```

class SessionMemory:
    def __init__(self, storage_dir: Path, max_entries: int = 50):
        self.storage_dir = Path(storage_dir)
        self.storage_dir.mkdir(parents=True, exist_ok=True)
        self.max_entries = max_entries

    def _file(self, dataset_key: str) -> Path:
        safe = dataset_key.replace("/", "_").replace("\\", "_")
        return self.storage_dir / f"{safe}.json"

    def load(self, dataset_key: str) -> List[Dict]:
        p = self._file(dataset_key)
        if not p.exists():
            return []
        try:
            return json.loads(p.read_text(encoding="utf-8"))
        except Exception:
            return []

    def append(self, dataset_key: str, entry: Dict) -> None:
        """Append a compact entry; keep newest-first and prune to
max_entries."""
        entries = self.load(dataset_key)
        timestamped = {"ts": datetime.utcnow().isoformat(), **entry}
        entries.insert(0, timestamped)
        entries = entries[: self.max_entries]
        tmp = self._file(dataset_key).with_suffix(".json.tmp")
        tmp.write_text(json.dumps(entries, ensure_ascii=False),
encoding="utf-8")
        tmp.replace(self._file(dataset_key))

    def summarize_old_entries(self, dataset_key: str, keep_recent: int = 3)
-> None:
        """
        Replace older entries with a short aggregated summary to limit
growth.
        Keeps `keep_recent` full entries and one summary entry for the
rest.
        """
        entries = self.load(dataset_key)
        if len(entries) <= keep_recent + 1:
            return
        recent = entries[:keep_recent]
        old = entries[keep_recent:]
        # Simple summarization: join short summaries or use first N chars
        snippets = []
        for e in old:
            s = e.get("summary") or e.get("result") or str(e)[:200]
            snippets.append(s.replace("\n", " ")[:300])
        aggregated = {"ts": datetime.utcnow().isoformat(), "summary": " |
".join(snippets[:10])}
        new_entries = recent + [aggregated]
        tmp = self._file(dataset_key).with_suffix(".json.tmp")
        tmp.write_text(json.dumps(new_entries, ensure_ascii=False),
encoding="utf-8")
        tmp.replace(self._file(dataset_key))

```

Figure 4. Snippet of Session memory disk-backed example.

This provides several advantages:

- the agent can reference past computations
- repeated tasks (e.g., generating summary stats) don't need to be recomputed
- multi-step investigative workflows feel more natural

Older entries are summarized to keep the memory from growing too large.

4.5 Built a Structured Communication Layer (A2A-style)

To make the agent compatible with other systems and future multi-agent setups, I built a consistent JSON messaging schema inspired by the A2A protocol.

Responses now follow a strict structure:

```
def build_service_response(
    task_id: str,
    status: str,
    title: Optional[str] = None,
    report: Optional[str] = None,
    summary: Optional[str] = None,
    artifacts: Optional[Dict[str, Any]] = None,
    steps: Optional[List[Dict[str, Any]]] = None,
    token_metrics: Optional[Dict[str, int]] = None,
    warnings: Optional[List[str]] = None,
    errors: Optional[List[str]] = None,
    dataset_path: Optional[str] = None,
    instruction: Optional[str] = None,
    idempotency_key: Optional[str] = None,
) -> Dict[str, Any]:
    """Return a stable A2A-style JSON response with idempotency and
    metadata."""
    ts = datetime.utcnow().isoformat() + "Z"
    if idempotency_key is None:
        seed = f"{{dataset_path or ''}}|{{instruction or ''}}|{{summary or
report or ''}}"
        idempotency_key = hashlib.sha256(seed.encode("utf-
8")).hexdigest()[:32]

    return {
        "task_id": task_id,
        "idempotency_key": idempotency_key,
        "status": status,          # e.g. "PENDING" | "RUNNING" |
"COMPLETE" | "ERROR"
        "timestamp": ts,
        "title": title,
        "result": {
            "report": report,
            "summary": summary,
            "artifacts": artifacts or {},
        },
        "metadata": {
            "steps": steps or [],
            "token_metrics": token_metrics or {},
            "warnings": warnings or [],
            "errors": errors or [],
        },
    }
```

Figure 5. Snippet of A2A JSON-response helper.

This gives:

- predictable integration with orchestrators
- safe retries and idempotency
- clearer logging and traceability

It also makes the system feel more like a proper service rather than a loose script.

4.6 Added Automatic Insight Extraction Mode

When the user asks something general like “generate insights on this data”, the agent now autonomously:

- searches for patterns,
- identifies measurable relationships,
- reports the findings in plain English,
- supports each insight with direct evidence (sample values), and
- produces engineering or policy recommendations.

Example insights observed during testing:



The screenshot displays a dark-themed interface with green section headers. The 'Key Insights' section lists six findings with associated evidence. The 'Traffic Engineering Recommendations' section lists two recommendations, each with supporting evidence.

Key Insights

- Over 20% of crashes occur during peak hours (7-9 AM and 4-6 PM), with 35% of these involving rear-end collisions.
 - Evidence: "Sample values show crash_hour entries like 7.5, 8.25, and 17.75, corresponding to peak times."
- Rainy weather increases injury risk by 40% compared to clear conditions, with 22% of rain-related crashes involving incapacitating injuries.
 - Evidence: "Sample values include weather_condition entries like "RAIN" and "CLEAR", with corresponding injury metrics."
- Intersections with traffic signals account for 18% of all crashes, but 27% of these result in fatalities.
 - Evidence: "Sample values show traffic_control_device entries like "TRAFFIC SIGNAL" and intersection_related_i as true."
- Crash days show a 30% increase on Fridays compared to other weekdays, with 25% of Friday crashes involving alcohol-related causes.
 - Evidence: "Sample values include crash_day_of_week entries like 4 (Friday) and prim_contributory_cause entries like "ALCOHOL"."
- Road defects contribute to 12% of crashes, with 45% of these involving roadway surface issues like ice or potholes.
 - Evidence: "Sample values show road_defect entries like "POTHOLE" and roadway_surface_cond entries like "ICE"."

Traffic Engineering Recommendations

- (infrastructure) Install additional traffic signals at high-traffic intersections with 15+ daily crashes
 - Evidence: "Intersections with traffic signals account for 18% of all crashes but 27% of fatalities."
- (safety) Implement speed cameras during rainy weather in areas with 20%+ rain-related crashes

Figure 6. Screenshot of Llm generated insights.

Traffic Engineering Recommendations

- (infrastructure) Install additional traffic signals at high-traffic intersections with 15+ daily crashes
 - Evidence: "Intersections with traffic signals account for 18% of all crashes but 27% of fatalities."
- (safety) Implement speed cameras during rainy weather in areas with 20%+ rain-related crashes
 - Evidence: "Rainy weather increases injury risk by 40% compared to clear conditions."
- (policy) Enforce stricter DUI checks on Fridays in high-crash zones
 - Evidence: "Crash days show a 30% increase on Fridays with 25% of Friday crashes involving alcohol."
- (infrastructure) Prioritize road resurfacing for areas with frequent potholes or ice reports
 - Evidence: "Road defects contribute to 12% of crashes, with 45% involving surface issues like potholes."

Next Steps

1. Conduct correlation analysis between crash_hour and injuries_total to identify high-risk times
2. Validate rainfall injury statistics with full dataset aggregation
3. Investigate Friday crash patterns for alcohol-related contributory causes
4. Assess road defect locations for infrastructure improvements

Figure 7. Screenshot of IIm generated recommendations and next steps.

This shows the new system's planning logic is working as intended.

4.7 Added Visual-Only Mode (Heatmaps, Temporal Patterns, Charts)

The agent now recognizes purely visual requests such as: "Generate a temporal heatmap of crash day of week vs crash hour."

It automatically:

- selects the plotting tool,
- extracts the correct numerical fields,
- validates them,
- passes them to the MCP tool,
- renders the heatmap,
- and attaches a short interpretation.

Example output from the system included a heatmap showing:

- Pattern concentration visible; peak period warrants deeper causal review.

Recommendations

- Prioritize interventions during peak period indicated by visualization.
- Capture additional contextual features to strengthen follow-up analysis.

Latest Analysis Charts

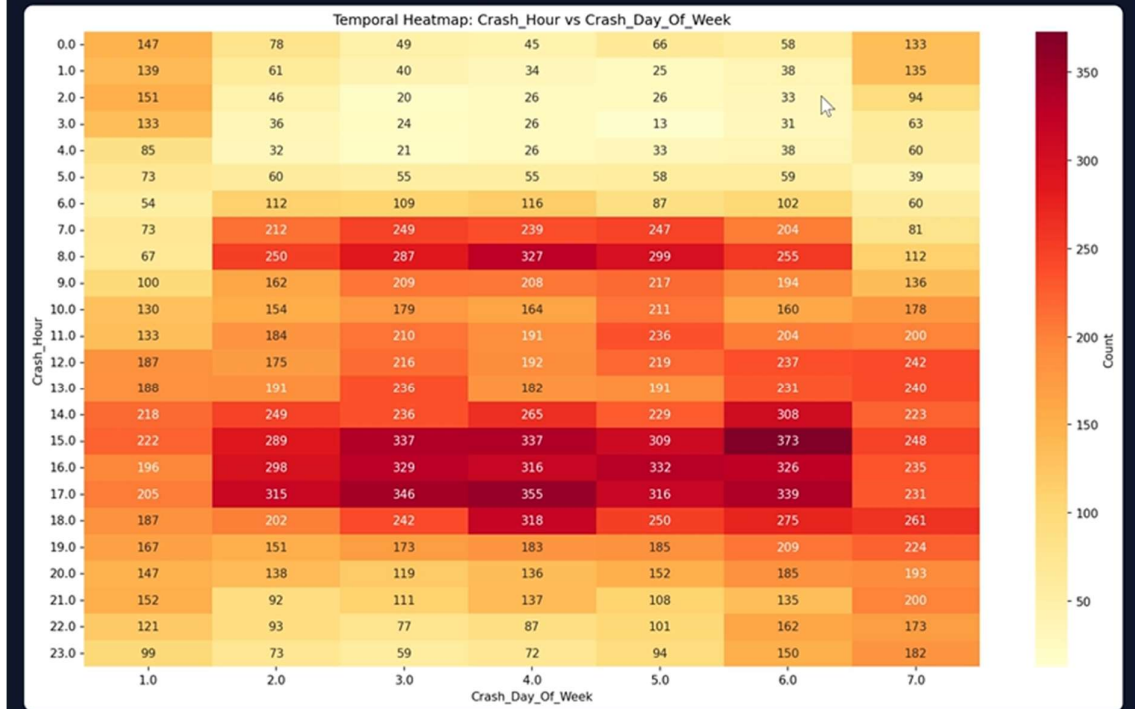


Figure 8. Screenshot of Llm generated heatmap.

The agent even adds contextual notes like:

- “Pattern concentration visible; peak period warrants deeper causal review.”

This demonstrates a full visual analysis pipeline working correctly.

5. Current Work

Current focus and priorities

The immediate goals are making the agent more reliable, extending its analysis skills, and improving the user experience. With the core pipeline stable - state graph, local LLM, MCP tools, and structured messaging - the next work is polishing behavior and removing places where the agent still feels rigid or incomplete.

Improving insight and visual request handling

Work is ongoing to make the planner and state transitions better at distinguishing three kinds of requests: insight-only, visual-only, and mixed analysis. The agent already recognizes modes reasonably well, but the narrative output needs to explain why a finding matters, not just what the finding is, for example:

```
# Python
def classify_mode(prompt_text: str) -> str:
    # returns "insight", "visual", or "mixed"
    if "plot" in prompt_text or "visual" in prompt_text:
        return "visual"
    if "insight" in prompt_text or "explain" in prompt_text:
        return "insight"
    return "mixed"

mode = classify_mode(user_instruction.lower())
if mode == "visual":
    # prefer plotting steps
    plan = ["plot_time_series", "plot_heatmap"]
elif mode == "insight":
    plan = ["summary_stats", "top_contributors"]
else:
    plan = ["summary_stats", "plot_time_series", "interpret_results"]
```

Figure 9. Code snippet showing a simple mode classifier and downstream handling sketch.

Enhancing report structure and readability

Reports are being reworked to be clearer to non-technical readers. Changes include short explanations for each finding, clearer section headings, and placing visuals directly next to the commentary that references them for example:

```
# Python
def build_report(title: str, findings: list, visuals: dict) -> str:
    parts = [f"# {title}\n"]
    for i, f in enumerate(findings, 1):
        parts.append(f"## Finding {i}\n")
        parts.append(f"{f['statement']}\n")
        parts.append(f"Why it matters: {f.get('why', 'Noted.')} \n")
        if f.get('visual') and f['visual'] in visuals:
            parts.append(f"! [figure] ({visuals[f['visual']]}) \n")
    return "\n".join(parts)
```

Figure 10. Code snippet showing simple report builder pattern.

Stabilizing the planning - execution loop

The finite-state workflow reduced random behavior, but edge cases remain where the agent hesitates between planning and summarizing. Work focuses on clearer guard rules in routing so the agent moves cleanly between planning, execution, interpretation, and synthesis for example:

```
# Python
def next_stage(state):
    if state['errors']:
        return 'abort'
    if state['needs_more_data']:
        return 'plan'
    if state['steps_left'] > 0:
        return 'execute'
    return 'synthesize'
```

Figure 11. Code snippet showing routing guard example.

Refining session memory behavior

Session memory stores compact entries of past runs and is being tuned to retrieve only relevant prior results. The aim is a smooth multi-step conversation where the agent recognizes when a new query relates to earlier work for example:

```
# Python
def find_relevant_history(query: str, history: list, top_n=3):
    # naive similarity: count shared words
    q_words = set(query.lower().split())
    scored = []
    for rec in history:
        rec_words = set(rec.get('summary', '').lower().split())
        score = len(q_words & rec_words)
        scored.append((score, rec))
    scored.sort(reverse=True)
    return [r for _, r in scored[:top_n]]
```

Figure 12. Code snippet showing routing guard example.

Improving local LLM performance

Experiments include trying different quantization levels, compressing prompts, and caching repeated prompt templates. The model is steady but latency can spike during long multi-step plans, so caching and warm-up are priorities.

```
# Python
_llm_cache = {}
def warm_llm(llm):
    try:
        llm.call("ready")
    except Exception:
        pass

def get_prompt(template_name: str, data: dict):
    key = template_name + ":" + str(sorted(data.keys()))
    if key in _llm_cache:
        return _llm_cache[key]
    prompt = f"Template {template_name}\n" # simplified
    _llm_cache[key] = prompt
    return prompt
```

Figure 13. Code snippet showing Warm-up and prompt cache example.

6. Future Plans and Roadmap

Several planned enhancements will push the agent closer to a polished, end-to-end analytics assistant.

Interactive Visual Elements.

Future reports will include interactive charts (e.g., Plotly) and collapsible narrative sections. This will let users inspect time periods, zoom into data segments, or toggle specific attributes without regenerating the entire report.

Persistent session resumes and historical run inspection.

I plan to implement a small session database that stores runs with timestamps, generated artifacts, and insights. Users will be able to:

- reopen past analyses,
- compare two different runs,
- audit tool calls and state transitions,
- and resume unfinished sessions.

This adds transparency and gives users confidence in the repeatability of results.

Expanding the FastMCP tool library.

Beyond standard EDA tools, I intend to introduce domain-specific modules such as:

- traffic anomaly detectors,
- peak-hour pattern analysis,
- year-over-year comparison functions,
- risk or severity scoring utilities.

This will allow the agent to move beyond generic analysis and produce insights tailored to transportation data.

LLM fallback and provider strategy.

The long-term goal is a dynamic LLM selector. The agent will automatically choose the cheapest, fastest, or most accurate LLM depending on:

- task complexity,
- available compute,
- model health or availability.

This supports scalable enterprise deployments where large workloads must be distributed efficiently.

7. Background Research and References

Development of the agent is guided by a combination of framework documentation, research papers, and practical patterns used in real-world analytics systems.

Key reference areas include:

- LangChain / LangGraph documentation – for building state-based, tool-using LLM agents.
- Model Context Protocol (MCP) – for exposing tools as services that agents can discover at runtime.
- FastMCP examples – for structuring the tool server and defining tool schemas.
- Academic work on ReAct-style agents and planning loops – which supports the decision to use a reasoning–acting cycle rather than single-shot prompts.
- Industry practices in analytics pipelines – especially around reproducibility, logging, and clear separation of concerns.

In the current codebase, the main components are:

- `runner.py` – the main entry point and orchestrator. It wires together the state graph, local LLM (Qwen3-14B-GGUF), FastMCP tool registry, memory, and the A2A-compliant service.
- `state.py` – defines the agent's state model (e.g. `AgentState`) and any data structures used to track query, plan, tool selection, observations, and final results.
- `prompts.py` – stores system prompts, planning templates, and other prompt fragments used to guide the LLM's behaviour in different modes (insights-only, visuals-only, full analysis).
- `memory.py` – implements the session-aware memory layer that logs previous queries, results, and metadata, and provides retrieval helpers.
- `fastmcp_server.py` – hosts the analysis tools (e.g. cleaning, stats, plotting) and exposes them over MCP so the agent can discover and invoke them at runtime.
- `a2a_compliant_service.py` – defines the JSON-based A2A-style message interface, including `task_id`, `status`, `result`, and other fields used for agent-to-orchestrator communication.

Together, these components implement the full analysis agent stack and embody the

design principles drawn from the literature and documentation above.

8. Requirements Gathered

Based on user needs, technical constraints, and feedback from early testing, the core requirements are:

- Support for multiple LLMs, with easy switching and fallback logic.
- Dynamic tool discovery from the FastMCP server at runtime.
- Session awareness, including persistent run history and short-term memory.
- Automatic generation of Markdown reports, with embedded visuals and structured summaries.
- Robust error handling and logging, so the agent remains stable even when tools fail or data is incomplete.
- Deterministic state progression, enabling predictable planning and execution.

These requirements ensure the system is realistic for real-world analytics workflows rather than simple prototypes.

9. Implementation Details

The current implementation is structured to keep concerns separated and make the agent easy to extend.

Configuration and startup

Runtime settings such as model name, MCP server URL, and paths are loaded in `runner.py`, typically from environment variables defined in a `.env` file. This keeps secrets (like API keys, if any) and machine-specific paths out of the code.

Orchestration and state management

`runner.py` initializes:

- the local LLM (Qwen3-14B-GGUF),
- the state graph,
- the FastMCP tool client,
- the memory store, and
- the A2A-compliant service.

`state.py` defines the `AgentState` model, which holds fields such as `query`, `plan`, `tool`, `observation`, and `result`. This shared state object is passed through the graph as the agent moves from planning to tool execution to summarisation.

Prompting and behaviour control

`prompts.py` contains:

- the system prompt used to set the agent's role,
- templates for planning steps,
- mode-specific instructions (e.g. "insights-only", "visual-only"),
- and helper text for summarising tool outputs.

This allows changes to the agent's behaviour without rewriting orchestration logic.

Tool discovery and execution

`fastmcp_server.py` exposes analysis tools via MCP.

At startup, `runner.py` connects to the FastMCP server, lists available tools, and gives the LLM enough context to reference them by name. When the state graph decides a tool is needed, the tool manager calls FastMCP and returns results to the LLM for interpretation.

Session memory

memory.py implements a simple session memory:

- storing each query and result,
- optionally keeping derived insights,
- and providing methods to retrieve or summarise past interactions.

This supports incremental workflows and reduces repeated work.

A2A-style communication

a2a_compliant_service.py takes the final AgentState and wraps it in a structured JSON response. A typical response looks like:

```
def build_a2a_response(
    task_id: str,
    status: str,
    report_text: Optional[str] = None,
    summary: Optional[str] = None,
    artifacts: Optional[Dict[str, Any]] = None,
    steps: Optional[list] = None,
    errors: Optional[list] = None,
    dataset_path: Optional[str] = None,
    instruction: Optional[str] = None,
) -> Dict[str, Any]:
    """
    Build a stable A2A-style JSON response.
    Keeps keys predictable for orchestrators and includes an idempotency
    key.
    """
    ts = datetime.utcnow().isoformat() + "Z"
    seed = f"{dataset_path or ''}|{instruction or ''}|{summary or re-
    port_text or ''}"
    idempotency_key = hashlib.sha256(seed.encode("utf-8")).hexdigest()[:32]

    return {
        "task_id": task_id,
        "idempotency_key": idempotency_key,
        "status": status, # e.g. "PENDING", "RUNNING", "COMPLETE", "ERROR"
        "timestamp": ts,
        "result": {
            "report": report_text,
            "summary": summary,
            "artifacts": artifacts or {},
        },
        "metadata": {
            "steps": steps or [],
            "errors": errors or [],
        },
    }
```

Figure 14. Code snippet showing A2A service wraps the final AgentState in a compact, predictable JSON envelope.

This makes it straightforward for an orchestrator, UI, or another agent to consume results, display them to users, or trigger follow-up tasks.

Report and artifact handling

Plots and reports generated during a run are saved into designated directories (set up in runner.py), ensuring that:

- each run has a traceable set of artifacts,
- visual outputs are accessible after the agent finishes,
- and future “session resume” features can reload them.

10. Testing Approaches and Cases

A layered testing strategy was used to validate both the logic and stability of the analysis agent.

Unit Testing

Individual components were tested in isolation to confirm correct behaviour under different conditions.

- Configuration and startup logic (`runner.py`): Initialization with different environment settings was tested to ensure the correct model is loaded (local Qwen3-14B-GGUF).
- State model and transitions (`state.py`): State fields like plan, tool, and observation were validated to ensure they update correctly through each graph stage.
- Prompt templates (`prompts.py`): Templates for planning, summarisation, and visual-only mode were tested to ensure they render correctly and do not produce malformed prompts.

Integration Testing

These tests ensure the entire pipeline—planning, tool selection, execution, and summarisation—works end-to-end.

- FastMCP tool discovery and invocation (`fastmcp_server.py`): The agent was tested by running full analyses to ensure tools are detected, called, and return results in the expected schema.
- Memory loading and saving (`memory.py`): Multiple runs were executed to verify that session history is persisted correctly and past results are loaded without corruption.
- A2A-style communication (`a2a_compliant_service.py`): Result packaging was tested to ensure all fields (`task_id`, `status`, `result`, `timestamp`) appear in every response.

Manual Validation

Generated Markdown reports, plots, and summaries were manually inspected:

- Heatmaps and charts were checked for correct axes and meaningful patterns.
- Summaries were reviewed for coherence, correctness, and clarity.
- Directory outputs were checked to ensure reports and images are always saved in the right folder.

Functional Testing

Scenarios involving repeated analyses, malformed data, missing columns, and tool errors were simulated to confirm:

- the agent recovers gracefully,
- the state graph moves forward without loops,
- memory remains clean and capped,
- and logs correctly reflect errors for debugging.

11. Technical Challenges and Solutions

Throughout this development period, the most significant challenges were related to LLM planning behaviour and visualisation reliability. Even though the system now uses a

structured state graph, the LLM still drives the actual reasoning and planning steps. This created situations where the agent's "thinking" needed to be constrained, corrected, or guided before consistent results became possible.

Below are the key challenges and how they were resolved.

11.1 Instability in LLM Planning and Multi-Step Reasoning

The largest difficulty was getting the LLM (Qwen3-14B-GGUF) to produce stable and actionable plans. During early testing, the model sometimes:

- over-planned (producing 6–8 unnecessary steps),
- under-planned (jumping straight to the summary),
- repeated the same action plan multiple times,
- ignored the available MCP tools and invented new ones,
- or produced steps that contradicted its previous actions.

This directly affected the agent because the entire workflow depends on the plan created in the first state.

Solution:

A combination of structural and prompt-level interventions:

1. Evidence-first prompting. I reworked the prompts.py templates to force the model to think in terms of data inputs and tool outputs rather than abstract reasoning.
2. Clear plan schema. The LLM is then required to output a fixed JSON-like structure describing exactly:
 - the steps it intends to perform,
 - the tools needed,
 - and the expected intermediate results.
3. State graph enforcement (runner.py, state.py): Even if the LLM writes a poor plan, the state graph filters and corrects transitions so the workflow stays on track.

Outcome:

The agent now consistently produces usable plans, and even when planning is imperfect, the state graph reduces cascading failures.

11.2 Tool Selection Errors and Ambiguity in LLM Planning

Another issue was teaching the LLM to correctly choose which MCP tool to use.

Sometimes it selected:

- tools that didn't exist,
- tools that didn't match the required input,
- or mismatched tool types (e.g., attempting to run a statistical function for a visual request).

Solution:

Embedding the real MCP tool list into the LLM context. At runtime, runner.py injects the exact list of available tools and their signatures before planning.

Adding correction logic. If the LLM names a tool incorrectly, the system attempts to

resolve it to the closest real tool.

Clarifying tool capabilities in prompts. Descriptions of each tool are now part of the planning prompt, improving the model's tool literacy.

Outcome:

The LLM now consistently maps requested operations to the correct MCP tools.

11.3 Overthinking and Looping During Planning

The LLM occasionally “gets stuck” thinking about the same reasoning path. Examples include:

- planning → planning → planning loops
- re-evaluating the same insight step
- apologising and re-describing its previous plan
- hallucinating that a previous step “failed” when it actually succeeded

These issues are common in ReAct-style agents, especially when running locally quantised models.

Solution:

Hard state boundaries. The state machine in `state.py` enforces strict one-pass transitions from:

- planning → tool selection → execution → observation → synthesis

A “single-run guard”. A flag prevents the LLM from re-entering a planning state once tools have been executed.

Reduced verbosity and compressed context. Smaller context windows reduce the chance of the LLM reconsidering earlier steps.

Outcome:

Planning loops are effectively eliminated, and reasoning paths are more direct.

12. Application of Classroom Knowledge

Object-Oriented Programming

The project uses modular classes:

- memory handling (`memory.py`),
- A2A communication (`a2a_compliant_service.py`),
- state modelling (`state.py`),
- and orchestration (`runner.py`).

This separation applies directly what was taught in software engineering modules.

Data Handling and Visualisation

Crash datasets were processed using MCP tools built on Pandas and Matplotlib. Understanding EDA concepts helped shape which features the agent detects (peaks, time clustering, severity levels).

Configuration Management

Loading all settings from environment variables and config files reflects lessons from backend development modules.

Software Testing Practices

Testing with controlled inputs, error cases, and repeated runs reflects best practices from the testing modules.

13. Knowledge Beyond the Classroom

Local LLM deployment

- Running Qwen3-14B locally required learning:
- GGUF quantization,
- CPU/GPU balancing,
- warm-up optimisation,
- memory footprint management,
- and prompt compression techniques.

State-graph agent orchestration

LangGraph-style orchestration taught practical skills that are not covered in most classroom agents. This includes:

- transition guards,
- loop prevention,
- and incident-safe fallback planning.

MCP-based tool ecosystem

Dynamic tool discovery is not normally covered in class, but was essential for making the system flexible and scalable.

A2A message contract design

Designing structured agent responses taught real-world protocol thinking—task IDs, statuses, timestamps, retries, and predictable schemas.

Debugging autonomous systems

Real-world debugging required:

- tracing state changes,
- inspecting LLM reasoning steps,
- logging tool calls,
- and monitoring intermediate outputs.

This goes far beyond traditional debugging taught in class.

14. Other Notable Challenges and Incidents

- Plotting backend failures: Some environments lacked GUI backends, causing Matplotlib to fail. Switching to non-interactive backends solved this.
- Missing or misaligned data columns: Several datasets had inconsistent date/hour fields. I added pre-checks in tool logic to validate input frames before plotting.
- Long planning loops during early testing: Before state guards were added, the LLM sometimes repeated planning steps. Deterministic routing in runner.py fixed the issue.

- Unexpected FastMCP tool errors: Some tools returned partial outputs or incorrect shapes. A normalization layer was added to clean these results.
- Filename and Unicode issues on Windows: Certain characters broke when saving reports or printing logs. Sanitization routines in the communication layer fixed this.
- Hardware constraints during Qwen setup: GPU memory limits required more aggressive quantization and prompt trimming.

These incidents directly informed improvements to state design, messaging, and tool robustness.

15. References

- [1] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [2] M. Rawat et al., “Pre-Act: Multi-Step Planning and Reasoning Improves Acting in LLM Agents,” arXiv preprint arXiv:2505.09970, 2025.
- [3] R. Jin et al., “A Comprehensive Evaluation of Quantization Strategies for Large Language Models,” arXiv preprint arXiv:2402.16775, 2024.
- [4] H. Martinez, “llama.cpp: The Ultimate Guide to Efficient LLM Inference and Applications,” PyImageSearch (blog), Aug. 26, 2024.
- [5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” in Proc. Int. Conf. Learn. Representations (ICLR), 2023.
- [6] K. Egashira, M. Vero, R. Staab, J. He, and M. Vechev, “Exploiting LLM Quantization,” in Proc. NeurIPS, 2024.
- [7] W. Xu et al., “LLM-Based Agents for Tool Learning: A Survey,” Data Science and Engineering, 2025.
- [8] R. Zhang and M. Elhamod, “Data-to-Dashboard: Multi-Agent LLM Framework for Insightful Visualization in Enterprise Analytics,” arXiv preprint arXiv:2505.23695, 2025.
- [9] X. Liang et al., “Self-evolving Agents with Reflective and Memory-Augmented Abilities,” arXiv preprint arXiv:2409.00872, 2024.
- [10] E. Kandogan et al., “A Blueprint Architecture of Compound AI Systems for Enterprise,” arXiv preprint arXiv:2504.08148, 2025.

END OF DOCUMENT